

Aer simulator fails to work with custom parametrized circuits and VQE

<https://github.com/Qiskit/qiskit/issues/10589>

ROW 75

New issue



- Qiskit Aer version: 0.12.2
- Python version: 3.11.3
- Operating system: Windows 10

When I try to execute the code below, I get the following error:

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_algorithms\minimum_eigensolvers\vqe.py", line 607, in _run_vqe
    estimator_result = job.result()
                        ^^^^^^^^^^^^^
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit\primitives\primitive_job.py", line 55, in result
    return self._future.result()
           ^^^^^^^^^^^^^^^^^^^
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\concurrent\futures\_base.py", line 181, in result
    return self._get_result()
           ^^^^^^^^^^^^^^^^^
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\concurrent\futures\_base.py", line 401, in _get_result
    raise self._exception
File "C:\Users\User\AppData\Local\Programs\Python\Python311\Lib\concurrent\futures\treadpool.py", line 58, in run
    result = self.fn(*self.args, **self.kwargs)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_aer\primitives\estimator.py", line 128, in compute
    return self._compute(circuits, observables, parameter_values, run_options)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_aer\primitives\estimator.py", line 188, in _transpile_circuits
    self._transpile_circuits(circuits)
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_aer\primitives\estimator.py", line 481, in _transpile_circuit
    circuit = self._transpile(circuit)
              ^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_aer\primitives\estimator.py", line 468, in _transpile_circuits
    return transpile(circuits, self._backend, **self._transpile_options._dict_)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit\compiler\transpiler.py", line 418, in compile
    out_circuits = pm.run(circuits, callback=callback)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

No one assigned

bug

No type

No milestone

None yet

No branches or pull requests


```

new_dag = pass_.run(dag)
#####

File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit\transpiler\passes\basis\unroll_custom_c
raise QiskitError(f"Error decomposing node {node.name}: {err}") from err
qiskit.exceptions.QiskitError: 'Error decomposing node hamiltonian: Unable to generate Unitary matrix for unbr

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
File "C:\Users\User\Desktop\MA-QAOA\temp\my_test.py", line 29, in <module>
    test_vqe()
File "C:\Users\User\Desktop\MA-QAOA\temp\my_test.py", line 25, in test_vqe
    result = vqe.compute_minimum_eigenvalue(-hamiltonian)
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_algorithms\minimum_eigsolvers\vqe.py'
optimizer_result = self.optimizer(
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_minimize.py", line 710, in mir
    res = _minimize_lbfgsb(fun, x0, args, jac, bounds,
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_lbfgsb_py.py", line 307, in _n
    sf = _prepare_scalar_function(fun, x0, jac=jac, args=args, epsilon=eps,
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_optimize.py", line 383, in _pr
    sf = ScalarFunction(fun, x0, args, grad, hess,
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_differentiable_functions.py",
    self._update_fun()
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_differentiable_functions.py",
    self._update_fun_impl()
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_differentiable_functions.py",
    self.f = fun_wrapped(self.x)
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\scipy\optimize\_differentiable_functions.py",
    fx = fun(np.copy(x), *args)
#####
File "C:\Users\User\Desktop\python3.11_venv\Lib\site-packages\qiskit_algorithms\minimum_eigsolvers\vqe.py'
raise AlgorithmError("The primitive job to evaluate the energy failed!") from exc
qiskit_algorithms.exceptions.AlgorithmError: 'The primitive job to evaluate the energy failed!'

```

Apparently, parametrized Hamiltonian gates fail to convert to unitaries when `AerEstimator` is used. However, if `qiskit.primitives.Estimator` is used instead, then everything works as expected. As far as I understand, one should be able to use these estimators interchangeably.

Steps to reproduce the problem

The following code reproduces the problem:

```

from qiskit import QuantumCircuit
from qiskit.circuit import ParameterVector
from qiskit.primitives import Estimator
from qiskit_aer.primitives import Estimator as AerEstimator
from qiskit.quantum_info import SparsePauliOp, Pauli
from qiskit_algorithms.minimum_eigsolvers import VQE
from scipy import optimize

def get_custom_ansatz(num_qubits: int) -> QuantumCircuit:
    params = ParameterVector('angles', num_qubits)
    circuit = QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        circuit.hamiltonian(Pauli('X'), params[i], [i])
    return circuit

def test_vqe():
    n = 12
    estimator = AerEstimator() # Estimator() works
    hamiltonian = SparsePauliOp.from_sparse_list([('ZZ', [0, 1], -1)], n)
    ansatz = get_custom_ansatz(n)
    optimizer = optimize.minimize
    vqe = VQE(estimator, ansatz, optimizer)
    result = vqe.compute_minimum_eigenvalue(-hamiltonian)
    return -result.eigenvalue.real

test_vqe()

```

What is the expected behavior?

I expect no errors.

Suggested solutions



Gaidailgor added **bug** on Aug 8, 2023



mtreinish transferred this issue from [Qiskit/qiskit-aer](#) on Aug 8, 2023



ikkoham on Aug 8, 2023

Contributor ...

Thanks. As [@mtreinish](#) transferred, this is the transpilation issue.

minimal code to reproduce this is

```
from qiskit import QuantumCircuit, transpile
from qiskit.circuit import ParameterVector
from qiskit_aer import AerSimulator

def get_custom_ansatz(num_qubits: int) -> QuantumCircuit:
    params = ParameterVector('angles', num_qubits)
    circuit = QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        circuit.hamiltonian(Pauli('X'), params[i], [i])
    return circuit

transpile(get_custom_ansatz(n), AerSimulator())
```



mtreinish on Aug 8, 2023

Member ...

It's actually more of an issue with the implementation of the [HamiltonianGate](#), it is raising errors when there is an unbound parameter when the transpiler is asking for it's definition to unroll it into terms that can be used for basis translation. There isn't a way to transpile it directly with an unbound parameter because it's definition can only be realized with a numeric parameter for `time` because the definition just returns a unitary gate, and we can't define the matrix without a numerical value for `time`.



Cryoris on Aug 15, 2023

Contributor ...

Yeah this only work with the `Estimator` in Qiskit (Terra) as it binds the parameter values before simulating the circuit.

If you'd like to use operator evolutions with the Aer Estimator, you can use the `PauliEvolutionGate` instead of the `QuantumCircuit.hamiltonian` function. The `PauliEvolutionGate` implements a Trotter expansion, so it is not exact, but this is what we would also have to run on a quantum computer.

I'll go ahead and close this issue, but feel free to re-open if the answers above didn't answer your question.



Cryoris closed this as **completed** on Aug 15, 2023